

Purple Team Outline

Week 10: API Authentication Attacks & Advanced Testing

Objective: To exploit flaws in modern authentication protocols like JWT and OAuth.

Task : JWT, OAUTH & API FUZZING

- **Description:** Analyze and attack JSON Web Tokens (JWT) used for session management. Test for misconfigurations in OAuth flows and perform automated fuzzing for hidden flaws.
- **Technical Requirements:**
 - Deconstruct a JWT into its Header, Payload, and Signature.
 - Identify common vulnerabilities: "None" algorithm attack, weak secret keys, or lack of signature verification.
 - Perform a token manipulation attack to escalate privileges to "Admin" status.
 - Conduct API fuzzing using custom wordlists to identify Broken Authentication or unhandled error states.

JWT (JSON WEB TOKENS):

Theory:

- What Vulnerable Web Application you are using & why ?
- Explain the three parts of a JWT and discuss why the payload is not considered secure even though the token is signed. How does signing differ from encryption in this context?
- JWTs are often used for stateless authentication. What are the advantages and disadvantages of using JWTs instead of traditional session-based authentication, especially in terms of scalability and security?
- Why is it important to set an expiration (**exp**) claim in a JWT? Describe how refresh tokens work alongside access tokens, and outline potential security risks if refresh tokens are compromised.
- What role does the signing algorithm (e.g., HS256 vs RS256) play in JWT validation? Compare symmetric and asymmetric signing methods and explain when you would choose one over the other.

Practical:

- Mention the vulnerable web application you are using (e.g., OWASP Juice Shop or DVWA) and provide a screenshot of it.
- Log in as a regular user on the web application, inspect the assigned JWT, decode it using JWT.io, provide screenshots of the token and decoded output, and clearly describe your findings along with their security impact – Provide the screenshot
- From the given JWT token, identify its three parts (header, payload, and signature) and explain the role and importance of each in creating a valid JWT – Provide the screenshot
- Attempt horizontal privilege escalation by modifying the JWT—specifically changing the **alg** value in the header and the **sub** value in the payload—then, if successful, explain in your own words how the privileges were escalated through token manipulation – Provide the screenshot

API FUZZING:

Theory:

- What Vulnerable Web Application you are using & why ?
- What is API fuzzing, and how do mutation-based and generation-based fuzzing techniques differ when testing API endpoints?
- What challenges arise when fuzzing authenticated or stateful APIs, and how can tokens (e.g., JWTs) and session handling impact fuzzing effectiveness?
- How can analyzing API responses (such as status codes, error messages, and response times) during fuzzing help identify potential security vulnerabilities?
- Compare the role of automated tools like Burp Suite Intruder and OWASP ZAP in API fuzzing. What are their strengths and limitations in discovering complex vulnerabilities?

Practical:

- Mention the vulnerable web application you are using (e.g., OWASP Juice Shop or DVWA) and provide a screenshot of it.
- Start inspecting any API endpoint you want to test, clearly mention the specific endpoint (URL) and the parameter you are going to fuzz, and briefly describe the purpose of fuzzing that parameter to identify potential input validation or security vulnerabilities – Provide the screenshot
- After inspecting the API, start performing **manual API parameter fuzzing** on parameters such as `id`, `author`, and `published`, then carefully analyze the response of each request to identify unexpected behavior, input validation issues, or potential security vulnerabilities – Provide the screenshot.
- Mention which API version is being used to communicate with the server (e.g., v0, v1, or v2) – Provide the screenshot
- After manual fuzzing, move towards automation using tools such as Burp Suite Intruder, ffuf, or wfuzz; mention which tool you used for automating the fuzzing process, specify the wordlist used, and in your own words explain why you chose that particular wordlist and tool for effective and efficient API fuzzing – Provide the screenshot
- After successful automated fuzzing, clearly mention all findings you discovered during testing and provide a concise summary of the results. Also explain what security risks were identified (if any) and describe their potential impact on the API or application.

OAUTH (Open Authorization):

Theory:

- What is OAuth and how does it allow users to grant third-party applications access to their data without sharing their password?
- What are the main roles in an OAuth 2.0 system (resource owner, client, authorization server, and resource server), and what is the responsibility of each?
- Explain the OAuth 2.0 Authorization Code Grant flow. Why is it considered more secure than the Implicit Grant flow?
- What are some common security risks in OAuth implementations (e.g., token leakage, redirect URI manipulation), and how can they be mitigated?
- What is the difference between an access token and a refresh token in OAuth 2.0, and why are both needed?